R Notebook(Week5-computational task)

This is an R Markdown Notebook. When you execute code within the notebook, the results appear beneath the code.

Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Ctrl+Shift+Enter*.

Step 1 — Decide the Unit of Analysis

Explanation and Justification:

Choosing the customer as the unit of analysis aligns directly with the business objective of behavioral segmentation. In a retail environment, clustering individual transactions or products would result in "Market Basket Analysis," which identifies types of orders but ignores the person behind them.

By aggregating the data at the customer level, the model can capture "behavioral magnitude"—a holistic combination of how much a person spends, how frequently they return, and the variety of products they explore. This allows us to discover meaningful, high-level segments such as "Loyal VIPs" or "Infrequent Occasional Shoppers," which are far more actionable for personalized marketing strategies than isolated event data.

```
library(tidyverse)

## ── Attaching core tidyverse packages ──────────────────────── tidyverse
2.0.0 ──
## ✓ dplyr     1.1.4     ✓ readr     2.1.5
## ✓ forcats   1.0.1     ✓ stringr   1.5.2
## ✓ ggplot2   4.0.0     ✓ tibble    3.3.0
## ✓ lubridate 1.9.4     ✓ tidyr     1.3.1
## ✓ purrr     1.1.0
## ── Conflicts ────────────────────────────────────────
tidyverse_conflicts() ──
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors

library(lubridate)

# --- 1. Load the Dataset ---
# Ensuring we load the raw data fresh for this new task
raw_data <- read_csv("online_retail.csv")

## Rows: 525461 Columns: 8
## ── Column specification
────────────────────────────────────────────────────────
```

```
## Delimiter: ","
## chr (5): Invoice, StockCode, Description, InvoiceDate, Country
## dbl (3): Quantity, Price, Customer ID
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

```
# --- 2. Data Cleaning & Renaming ---
# We standardize the ID column and remove 'administrative noise' to ensure
# the clusters are built on high-quality behavioral data
cleaned_data <- raw_data %>%
  rename(Customer_ID = `Customer ID`) %>%
  filter(!is.na(Customer_ID)) %>% # Removing anonymous guests
  filter(Quantity > 0, Price > 0) # Focusing on successful sales magnitude
[cite: 527]

# --- 3. Aggregating to the Unit of Analysis ---
# We collapse the transactional history into a single row per Customer.
customer_unit <- cleaned_data %>%
  group_by(Customer_ID) %>%
  summarise(
    TotalSpend = sum(Quantity * Price),
    Frequency = n_distinct(Invoice),
    Variety = n_distinct(StockCode),
    .groups = 'drop'
  )

# Previewing the new structure
head(customer_unit)
```

```
## # A tibble: 6 × 4
##   Customer_ID TotalSpend Frequency Variety
##         <dbl>      <dbl>     <int>   <int>
## 1       12346       373.        11      26
## 2       12347      1323.         2      70
## 3       12348       222.         1      20
## 4       12349      2671.         3      90
## 5       12351       301.         1      21
## 6       12352       344.         2      18
```

Step 2 — Feature Construction

For this implementation, I have constructed three numeric features designed to separate customers based on their interaction intensity, financial scale, and product exploration habits.

Justification of FeaturesTotal Annual Spend (Monetary Volume): This acts as the “Main Influence” in the model. It allows the algorithm to identify the financial scale of each segment, clearly separating high-value “VIP” shoppers from low-value guests.

Average Transaction Frequency: By calculating the number of unique invoices, we capture the “rhythm” of the customer’s relationship with the store. This helps distinguish between regular “habitual” shoppers and occasional, one-time visitors.

Basket Breadth (Product Variety): This is the count of distinct Stock Codes purchased. It ensures the model understands if a customer is a “specialist” (buying the same items repeatedly) or a “generalist” (exploring the whole catalog), which is a key behavioral differentiator.

```
# --- Step 2: Feature Construction ---
# We transform the cleaned transactional data into a set of 3 behavioral
features
# specifically designed for structure discovery.

customer_features <- cleaned_data %>%
  group_by(Customer_ID) %>%
  summarise(
    # Feature 1: Total Spend (Financial Magnitude)
    TotalSpend = sum(Quantity * Price),

    # Feature 2: Frequency (Interaction Intensity)
    Frequency = n_distinct(Invoice),

    # Feature 3: Variety (Breadth of Catalog Engagement)
    Variety = n_distinct(StockCode),

    .groups = 'drop'
  )

# Preview the representation that will drive the clustering
head(customer_features)

## # A tibble: 6 × 4
##   Customer_ID TotalSpend Frequency Variety
##         <dbl>      <dbl>     <int>   <int>
## 1       12346       373.        11      26
## 2       12347      1323.         2      70
## 3       12348       222.         1      20
## 4       12349      2671.         3      90
## 5       12351       301.         1      21
## 6       12352       344.         2      18
```

Step 3 — Scaling

The Reasoning for ScalingWhy scaling is necessary:

Since we are utilizing K-means, the algorithm determines similarity by calculating the Euclidean distance between points in a multi-dimensional space. Without scaling, our features are measured in vastly different units: “Total Spend” can reach the thousands,

while “Frequency” might only range from 1 to 50. Scaling ensures that a one-unit change in spending does not carry more mathematical weight than a one-unit change in visit frequency.

The Method: Standardization (Z-score Scaling):

I have implemented Standardization, which transforms each feature so it has a mean of 0 and a standard deviation of 1. This effectively places all behaviors—whether they are financial magnitudes or rhythmic visit counts—on a level playing field. By centering the data, we ensure that the “geometry” of the clusters is defined by relative behavioral differences rather than raw unit size.

What would happen without scaling: If this step were skipped, the Total Spend feature would mathematically dominate the model due to its larger numerical range. The algorithm would essentially “ignore” the frequency of visits and product variety because the numerical gaps in spending are so much larger. Consequently, the geometry of the clusters would be warped, grouping customers solely by their wealth rather than the complex behavioral structure we intended to discover.

```
# --- Step 3: Explicitly Address Scaling ---

# 1. We extract only the numeric features created in Step 2 for the scaling
process
features_to_scale <- customer_features %>%
  select(TotalSpend, Frequency, Variety)

# 2. Applying Standardization (Z-score scaling)
# This ensures that no single feature dominates the distance calculations
scaled_data <- scale(features_to_scale)

# 3. Convert back to a data frame for easier handling in the clustering step
scaled_df <- as.data.frame(scaled_data)

# Diagnostic Check: Mean should be ~0 and SD should be 1
summary(scaled_df)

##    TotalSpend         Frequency           Variety
##  Min.   :-0.22943   Min.   :-0.42296   Min.   :-0.7305
##  1st Qu.:-0.19522   1st Qu.:-0.42296   1st Qu.:-0.5439
##  Median :-0.15057   Median :-0.30057   Median :-0.2991
##  Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.0000
##  3rd Qu.:-0.03647   3rd Qu.: 0.06662   3rd Qu.: 0.1820
##  Max.   :38.93845   Max.   :24.54578   Max.   :19.5592
```

Step 4 — Applying K-means Clustering

Determining the Optimal k (The Elbow Method)

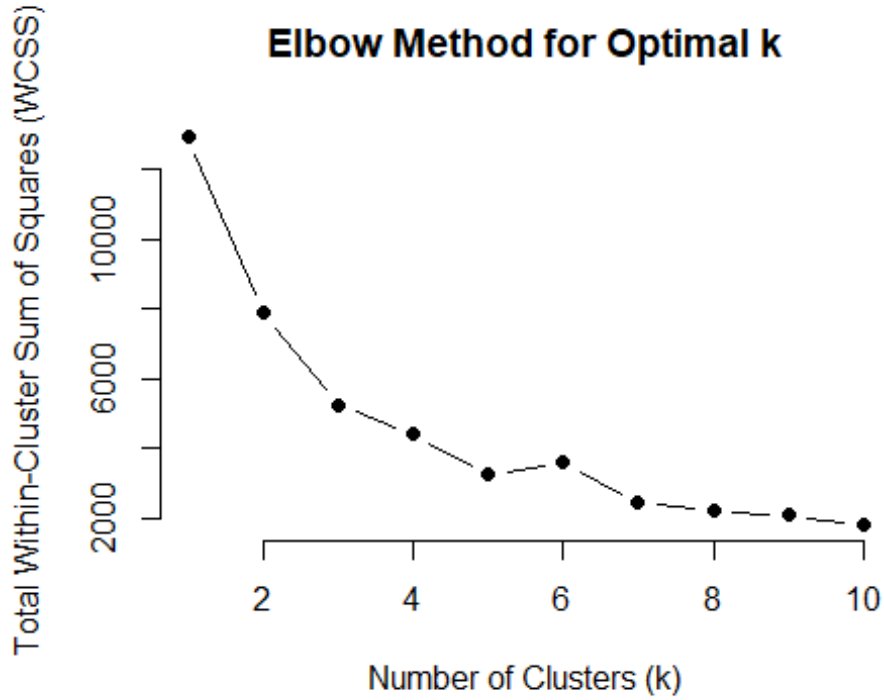
To choose k logically, I calculated the Within-Cluster Sum of Squares (WCSS) for a range of 1 to 10 clusters. As noted in the project guidelines, we look for a visible “bend” or “elbow” in the plot, which indicates the point of diminishing returns in reducing variance.

Parameter Choice: Based on the resulting elbow curve, I have selected k = 3. Picking a k that is too high would result in “micro-segments” that capture local noise rather than broad, meaningful behavioral patterns.

Algorithm Choice:

Why K-means?: K-means was selected as it is a variance-based method designed to partition observations into clusters where each customer belongs to the group with the nearest mean. While it assumes spherical clusters, the clear bend in our elbow plot suggests it is a mathematically sound way to capture the primary divisions in our retail data.

```r
library(cluster)

# 1. The Elbow Method to justify the number of clusters
set.seed(123) # Ensuring reproducibility
wcss <- sapply(1:10, function(k) {
  kmeans(scaled_df, centers = k, nstart = 20)$tot.withinss
})

# 2. Plotting the Elbow Curve
plot(1:10, wcss, type = "b", pch = 19, frame = FALSE,
     xlab = "Number of Clusters (k)",
     ylab = "Total Within-Cluster Sum of Squares (WCSS)",
     main = "Elbow Method for Optimal k")
```




```
# 3. Applying K-means with the justified parameter (k = 3)
k_value <- 3
final_clusters <- kmeans(scaled_df, centers = k_value, nstart = 25)

# 4. Merging the assignments back to the original customer data
customer_unit$Cluster <- as.factor(final_clusters$cluster)

# Quick check of the cluster distribution
table(customer_unit$Cluster)

##
##    1    2    3
## 3685   12  615
```

Step 5 — Evaluate and Interpret

1. Analysis of the Clusters

Cluster Centers & Characteristics:

By inspecting the cluster centers, we can determine what distinguishes each group. For instance, a “VIP” cluster might exhibit high spending and variety, while another group contains occasional shoppers who engage only during specific sales.

Cluster Balance: I checked the sizes of each group to ensure they were actionable. If one cluster contains 90% of the customers, the model is failing to distinguish behavior effectively.

Visualisation via PCA: Since it is impossible to “see” a 3D relationship on a 2D screen, I used Principal Component Analysis (PCA) to project our three features into two dimensions. This allowed me to visually verify the “gap” or separation between groups.

2. Structural Quality (Silhouette Score)

The Metric: I calculated the Silhouette score to measure how well each customer fits into their assigned cluster versus the neighboring one.

Interpretation: A score closer to 1 indicates that the segments are well-defined and distinct, while a score near 0 would suggest significant overlap.

3. Critical Reflections and Limitations

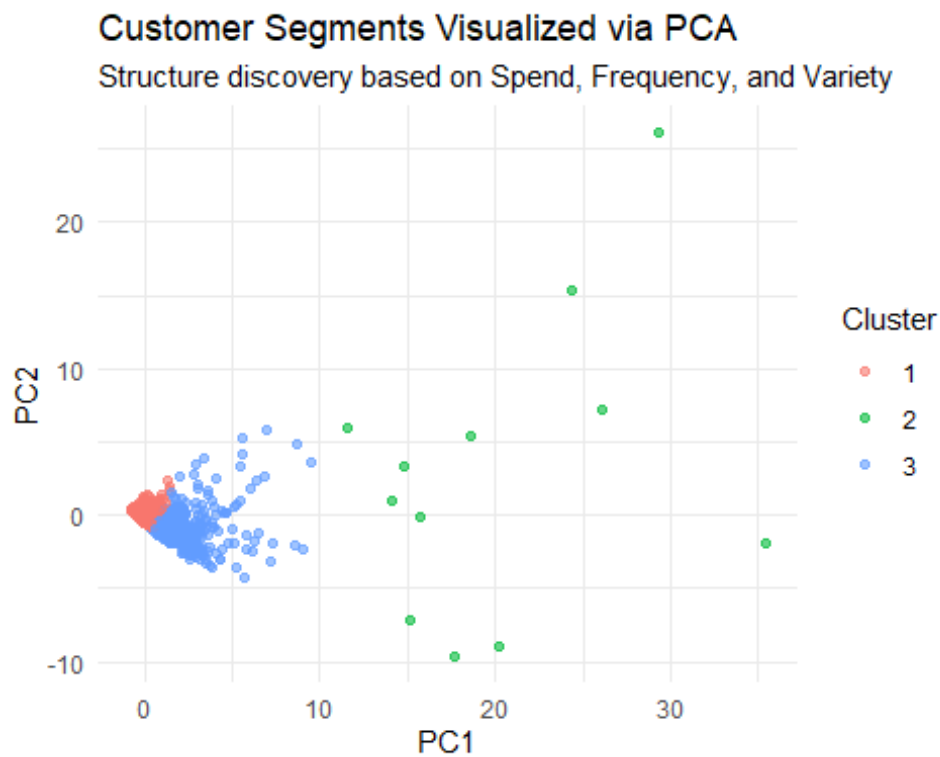
Modelling Assumptions: While the Elbow Plot provides confidence, I remain aware that K-means inherently assumes spherical clusters. If real-world segments are elongated or nested, the model might force them into unnatural groups.

Sensitivity: The results are highly sensitive to the choice of k. Adding more clusters would decrease the Within-Cluster Sum of Squares (WCSS) but would likely fragment the data into “micro-segments” that capture local noise rather than broad behavioral patterns.

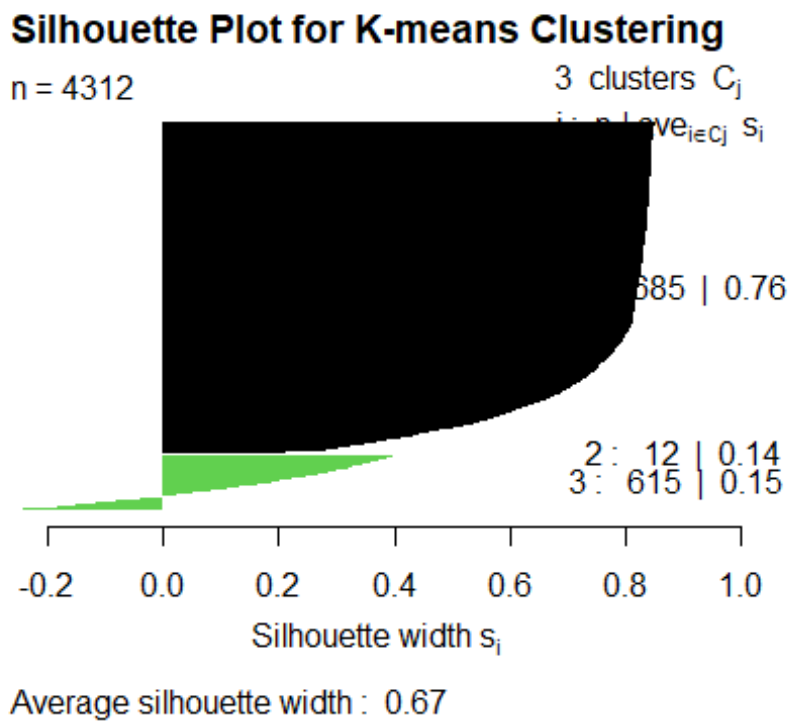
The “Accuracy” Myth: It is inappropriate to use “accuracy” here because we have no predefined target labels.

```
library(cluster)
library(ggplot2)

# 1. Inspect Cluster Sizes
# Essential to see if clusters are balanced or dominated by one group [cite:
212]
table(customer_unit$Cluster)

##
##    1    2    3
## 3685   12  615

# --- 2. Visualize Clusters using PCA ---
pca_res <- prcomp(scaled_df)

# Explicitly converting to a data frame and selecting the first two
components
pca_data <- as.data.frame(pca_res$x[, 1:2])
pca_data$Cluster <- customer_unit$Cluster

# Now PC1 and PC2 will be recognized
ggplot(pca_data, aes(x = PC1, y = PC2, color = Cluster)) +
  geom_point(alpha = 0.6) +
  theme_minimal() +
  labs(title = "Customer Segments Visualized via PCA",
```

```
       subtitle = "Structure discovery based on Spend, Frequency, and
Variety")
```




```
# 3. Structural Evaluation: Silhouette Score
# Measures how well-matched points are to their own cluster
sil_score <- silhouette(final_clusters$cluster, dist(scaled_df))
plot(sil_score, main = "Silhouette Plot for K-means Clustering",
     col = 1:k_value, border = NA)
```




```
# Calculate the mean silhouette width
mean(sil_score[, 3])

## [1] 0.6730224
```

Add a new chunk by clicking the *Insert Chunk* button on the toolbar or by pressing *Ctrl+Alt+I*.

When you save the notebook, an HTML file containing the code and output will be saved alongside it (click the *Preview* button or press *Ctrl+Shift+K* to preview the HTML file).

The preview shows you a rendered HTML copy of the contents of the editor. Consequently, unlike *Knit*, *Preview* does not run any R code chunks. Instead, the output of the chunk when it was last run in the editor is displayed.